



Duale Hochschule Baden-Württemberg Mannheim

Studiengang Wirtschaftsinformatik
Studienrichtung Data Science

Computer Vision zur Fernsteuerung von Powerpoint-Präsentationen

WWI22DSA

Machine Learning Projekt | Prof. Dr. Maximilian Scherer
Eingereicht 13.07.2025

Joel Bück	4860895
Lukas Runge	7590014
Martin Schauer	7961802
Lukas Stamm	8402366

Inhaltsverzeichnis

1	Einleitung	1
2	Use-Case	1
3	Grundlagen	2
3.1	Handdetektion	2
3.1.1	Mediapipe	6
3.2	Zeitreienklassifikation	7
3.2.1	ST-GCN	10
4	Umsetzung	10
4.1	Projektaufbau	11
4.2	Training	13
4.3	Modellarchitektur	14
4.4	Ergebnis	16
5	Ausblick	17
6	Fazit	17
	Literaturverzeichnis	19

1 Einleitung

Im Rahmen des Machine Learning Projekts möchten wir uns mit dem Teilbereich der *Computer Vision* befassen. Bei einer initialen Recherche und Ideensammlung fanden wir die *Mediapipe*-Modelle von Google[2]. Eines davon ermöglicht Echtzeittracking von Handpositionen. Ausgehend von diesem Modell entwickeln wir unsern Use-Case: Das Steuern von Powerpoint-Präsentation mithilfe von Handgesten. Dafür wird neben dem Google-Modell ein Zeitreihenklassifikationsmodell und eine Integration von Powerpoint mithilfe von Python benötigt.

2 Use-Case

Powerpoint-Präsentationen sind oft besser, wenn der Vortragende frei sprechen kann und die Folien urein unterstützend wirken. Um diesen Zweck zu erfüllen ist es hinderlich, den Computer manuell mit Tastendrücken zu bedienen. Dieses Problem möchten wir beheben. Deshalb ist der Use-Case, den wir bearbeiten möchten, die Digitalisierung von Präsentern. Presenter sind kleine Fernbedienungen, um eine Powerpoint-Präsentation steuern zu können, ohne direkt am PC zu stehen und die Tasten zu drücken. Das Ziel unseres Projekts ist es, diese Art der Steuerung nachzubilden und die Interaktion mithilfe von Handgesten darzustellen.

Da die geplante Anwendung während Präsentationen verwendet werden soll, ist eine Echtzeitverarbeitung sehr relevant. Gleichzeitig muss zuverlässig die richtige Geste identifiziert werden und von normalen Handbewegungen eines Vortrags unterschieden werden. Die Konsequenz von Fehlklassifizierungen ist, dass das Publikum verwirrt und der Vortragende abgelenkt wird. Letztendlich führt das zu einer schlechteren Vortragsqualität. Unser Use-Case sieht eine Kamera vor, die die Hand des Vortragenden klar sehen kann, diese beobachtet und im richtigen Moment einen der vordefinierten Befehle ausführt.

Das Problem spaltet sich demnach in mehrere Teilschritte:

- *Kamera aufnehmen* Der erste Schritt ist das Auslesen der Kamera.
- *Hände finden* Im zweiten Schritt müssen die Bounding-Boxen der Hände im aufgenommenen Frame gefunden werden.

- *Handskelett identifizieren* Innerhalb der Bounding-Boxes müssen die genauen Positionen der Handgesten gefunden werden. Aus den detektierten Händen muss die Hand gefunden werden, die die relevante Geste ausübt.
- *Zeitreihe klassifizieren* Jedes Zeitfenster an Skelettpositionen muss klassifiziert und einem Befehl zugeordnet werden. Die Gesten werden vor dem Training festgelegt. Es muss ein Mechanismus eingeführt werden, der reguläre Bewegungen von definierten Gesten unterscheidet.
- *Befehl in Powerpoint ausführen* Wenn ein valider Befehl gefunden wurde, muss dieser in der aktiven Powerpoint-Präsentation ausgeführt werden.

Für den Use-Case sind also zwei Modelle notwendig. Zunächst wird ein Modell zur Handskelett-Identifikation benötigt, das sowohl die Bounding-Boxes als auch die Skelettpunkte findet. Danach folgt ein klassifizierungsmodell, um die Befehle auszuwerten. Die restlichen Schritte können durch reguläre Algorithmen ausgeführt werden

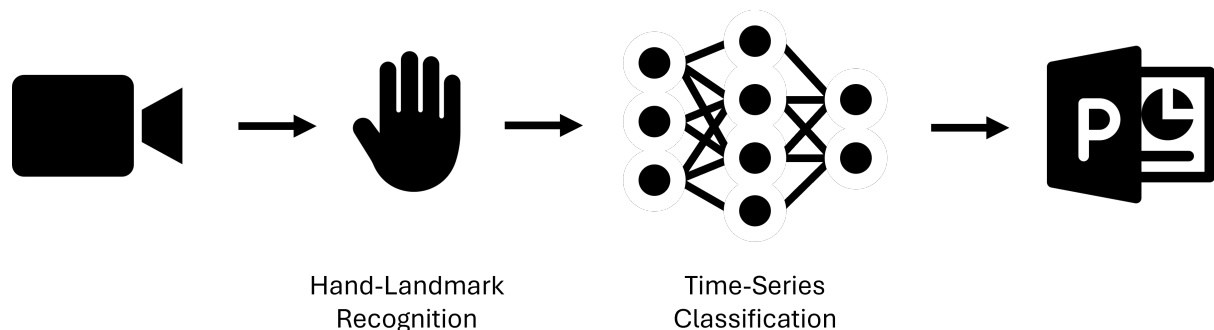


Abbildung 2.1: Diagramm der benötigten Modelle im Programmablauf. Zwischen der Kameraaufnahme und der Powerpointpräsentation muss zunächst die Hand detektiert werden und danach muss die Geste klassifiziert werden

3 Grundlagen

Im Folgenden wird kurz beleuchtet, welche Ansätze für die beiden Modelle in der Literatur und der Praxis eingesetzt werden.

3.1 Handdetektion

Die Handdetektion setzt sich aus zwei Schritten zusammen. Zuerst werden Regionen im Bild gefunden, in denen sich Hände befinden. Diese Regionen heißen Bounding-Boxes. Danach werden 21 Punkte im drei dimensionalen Raum gesucht, die zusammen mit einer festgelegten Adjazenzmatrix das Handskelett bilden (Abb. 3.1).

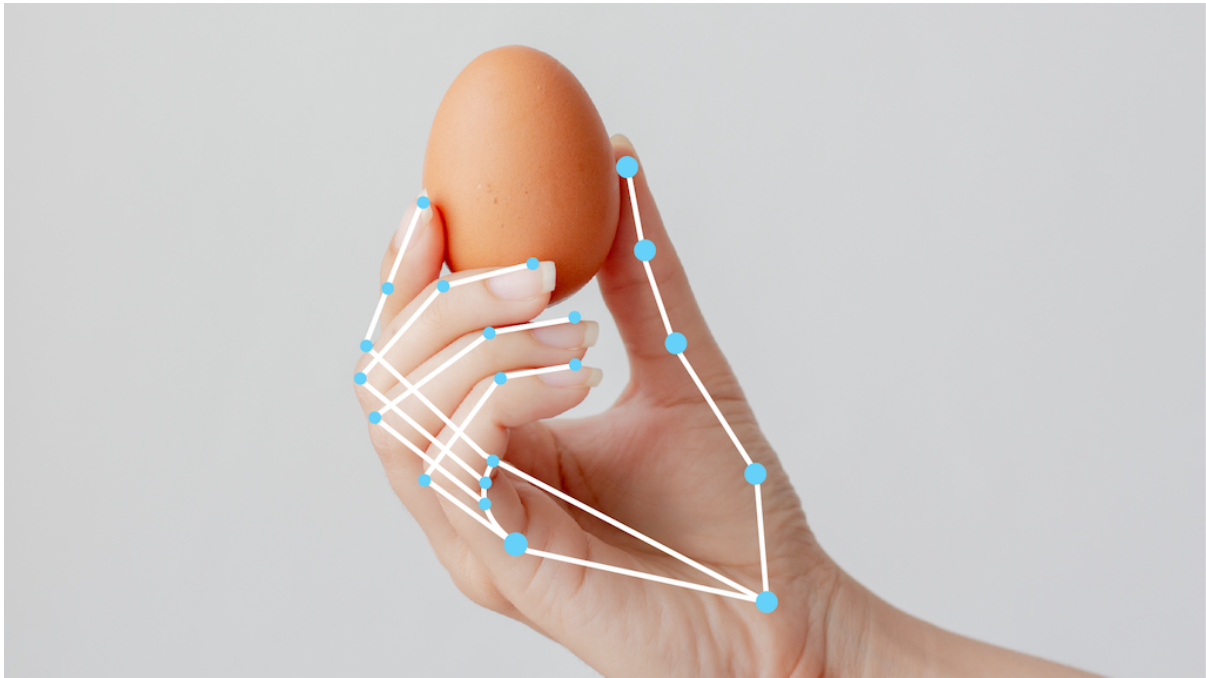


Abbildung 3.1: Alle 21 Landmarks mithilfe der Adjazenzmatrix verbunden [1]

Haar-Cascade-Klassifikator

Ein klassischer Ansatz zur Gesichtserkennung in Bildern basiert auf sogenannten Haar-Cascade-Klassifikatoren, die bereits im Jahr 2001 vorgeschlagen wurden.[20] Da Hände ebenfalls charakteristische visuelle Merkmale aufweisen, lassen sich entsprechende Klassifikatoren trainieren, die auf ähnliche Weise wie bei Gesichtern Bildregionen analysieren und bewerten. Besonders in Anwendungen wie Gestenerkennung, Mensch-Maschine-Interaktion oder der Steuerung von Benutzeroberflächen durch Handbewegungen kann diese Methode eine einfache und schnelle Lösung bieten.

Dieses Verfahren nutzt eine Kombination aus Feature-Extraktion und maschinellem Lernen, um Gesichter effizient zu detektieren. Zunächst werden aus Trainingsdaten eine große Anzahl von Merkmalen extrahiert - typischerweise etwa 160.000. Diese Merkmale ähneln einfachen Filtern, wie sie auch in Convolutional Neural Networks (CNNs) verwendet werden, und beschreiben beispielsweise Kanten, Linien oder Helligkeitsunterschiede in bestimmten Bildregionen.[20]

Da jedoch nicht alle dieser Merkmale gleich relevant für die Gesichtserkennung sind, kommt ein Verfahren namens AdaBoost zum Einsatz. Dieses wählt aus der großen Menge an Merkmalen eine kleinere, besonders aussagekräftige Teilmenge aus. Meistens zwischen 200 und 6000 Features. Diese ausgewählten Merkmale werden anschließend in einem Klassifikator verwendet, der systematisch über das Bild iteriert. Dabei wird jedes mögliche Fenster von 24x24 Pixeln untersucht, um zu bestimmen, ob es ein Gesicht enthält. Fenster, die eindeutig keine Gesichter zeigen, werden sofort verworfen, was den Suchraum erheblich reduziert. Eine wesentliche Verbesserung dieses Ansatzes ist die sogenannte Kaskadierung der Merkmale. Dabei werden die Merkmale in Gruppen organisiert, die nacheinander auf ein Bildfenster angewendet werden. Wenn eine Gruppe von Merkmalen ein Fenster als negativ klassifiziert, wird dieses sofort ausgeschlossen, ohne dass die restlichen Merkmale geprüft werden müssen. Diese Technik ermöglicht eine erhebliche Beschleunigung der Gesichtserkennung, da viele Bildbereiche frühzeitig aussortiert werden

können.[20]

Region based Convolutional Neural Networks

Region based Convolutional Neural Networks (R-CNN) basieren auf der Idee, zunächst potenzielle Regionen in einem Bild zu identifizieren, die Objekte enthalten könnten, und anschließend diese Regionen mithilfe eines CNN zu klassifizieren.

Der ursprüngliche R-CNN-Ansatz kombiniert ein CNN zur Merkmalsextraktion mit zwei separaten Modellen: einer Support Vector Machine (SVM), die für die Klassifikation der extrahierten Regionen zuständig ist, und einem linearen Regressor, der die Position der erkannten Objekte durch Feinjustierung der Bounding Boxes verbessert. Die CNN-Schichten bestehen aus Convolutional Layers, gefolgt von Aktivierungsfunktionen wie ReLU, um Nichtlinearitäten einzuführen und so die Lernfähigkeit des Netzwerks erhöhen.[3]

Trotz seiner hohen Genauigkeit ist der ursprüngliche R-CNN-Ansatz sehr rechenintensiv und langsam, da für jedes Region Proposal ein vollständiger Forward-Pass durch das CNN durchgeführt werden muss. Dies macht den Einsatz in Echtzeitanwendungen nahezu unmöglich. In der Folge wurden zahlreiche Verbesserungen vorgeschlagen, wie etwa Fast R-CNN und Faster R-CNN. Es gibt viele Weiterentwicklungen der R-CNN Architektur: Frühe Modelle sind beispielsweise LeNet, AlexNet und ZF Net. Ein Beispiel für ein modernes, tiefes Netzwerk ist ResNet. Diese Architekturen unterscheiden sich in ihrer Tiefe, der Anzahl der Parameter und der Effizienz, mit der sie Merkmale aus Bildern extrahieren.[3]

Hybride Ansätze

Ein Vorteil der CNN Architektur ist, dass in Hybriden Ansätzen Tiefensensoren hinzugefügt werden können, ohne die GRundarchitektur zu verändern. Durch die Tiefensensoren können neben den zweidimensionalen Features auch 3D-Koordinaten gefunden werden. Somit sind modifizierte CNNs zur räumlichen Lokalisierung von Händen interessant.[21]

OpenPose

OpenPose ist ein leistungsfähiges Verfahren zur Pose Estimation, das die vollständige Skelett-Erkennung mehrerer Personen in Echtzeit ermöglicht. Dabei werden sogenannte Confidence Maps verwendet, um einzelne Körperteile zuverlässig zu lokalisieren. Ergänzend kommen Part Affinity Fields (PAFs) zum Einsatz, die Informationen darüber liefern, wie diese Körperteile zueinander in Beziehung stehen. Diese PAFs helfen dabei, zusammengehörige Bildregionen zu identifizieren, was durch ein bipartites Matching-Verfahren auf Basis von Graphenstrukturen realisiert wird. Letztendlich werden die Einzelteile zu einem Ergebnis geparsed. Das Ergebnis sind präzise identifizierte und korrekt zusammenhängende Skelette der Personen im Bild.[4, 5, 16, 22]

Die technische Umsetzung erfolgt durch ein mehrstufiges CNN, das zunächst mit größeren 7×7 -Kernen arbeitet, um grobe Strukturen zu erfassen, und anschließend mit feineren 3×3 -Kernen, um Details zu verfeinern. Dieses Netzwerkdesign ermöglicht eine robuste und gleichzeitig effiziente Analyse von Bilddaten. Bemerkenswert ist, dass OpenPose

nicht nur auf Menschen beschränkt ist. Nach kleinen Modifikationen wurde es sogar erfolgreich auf Fahrzeuge angewendet, was die Vielseitigkeit des Ansatzes unterstreicht.[4, 5, 16, 22]

Das Mediapipe-Modell zur Hand-Landmark-Detektion verwendet einen ähnlichen Ansatz. Allerdings sind keine detaillierten Informationen zur genauen Architektur dieses Modells verfügbar. Beide Modelle eignen sich aufgrund ihrer Leistungsfähigkeit für Aufgaben mit Echtzeitanforderung.

Modelle für mobile Geräte

Beispiele für mobile-optimierte Modelle zur Handerkennung sind BlazePalm und MobileNetV3, die speziell für den Einsatz auf mobilen Geräten in Echtzeit entwickelt wurden.

BlazePalm basiert auf einer optimierten Architektur, die ein Encoder-Decoder-Design zur Feature extraction verwendet. Das Modell ist darauf ausgelegt, effizient auf mobilen Geräten zu laufen. Es verfolgt einen zweistufigen Ansatz: Zunächst wird die Handfläche erkannt, da dies schneller und ressourcenschonender ist als die vollständige Handerkennung. Die Handfläche wird mit einer schnelleren Methode getrackt. Erst wenn die Position der Handfläche nicht mehr zuverlässig erkannt werden kann, wird erneut eine Bounding-Box-Detektion durchgeführt. Anschließend erfolgt die präzise Landmark-Detektion innerhalb der erkannten Bounding-Box. Dieser Ansatz reduziert die Rechenlast erheblich und ermöglicht eine stabile und schnelle Handverfolgung und Gestenerkennung.[25]

MobileNetV3 setzt auf eine besonders effiziente Form von CNN. Die Architektur kombiniert leichte Standard-Convolutions mit 1x1-Convolutions, wobei letztere mit einer höheren Anzahl an Filtern arbeiten. Zusätzlich kommen lineare Bottlenecks und Residual-Verbindungen zum Einsatz, um die Modelltiefe zu erhöhen, ohne die Rechenleistung übermäßig zu beanspruchen. Diese Struktur ermöglicht eine hohe Genauigkeit bei gleichzeitig geringem Ressourcenverbrauch, was MobileNetV3 ideal für den Einsatz in mobilen Anwendungen macht.[10]

Beide Modelle zeigen, dass die Echtzeitanforderung ohne Einbuße an Genauigkeit bei der Handerkennung erfüllt werden kann.

Transformer

Ein eher unkonventioneller Ansatz für die Gestenerkennung bei Händen ist der Einsatz von Vision Transformers. Diese Technologie wurde ursprünglich für Aufgaben in der natürlichen Sprachverarbeitung (NLP) entwickelt. Dabei wird das Transformer-Modell, bekannt für seine Self-Attention-Mechanismen, auf Bilddaten übertragen und ermöglicht so eine leistungsstarke Alternative zu klassischen Bildverarbeitungsverfahren. Wie in NLP nutzen Vision Transformers das Konzept der Self-Attention, um nun Beziehungen zwischen verschiedenen Bildteilen zu modellieren. Dabei wird ein Bild in kleine Patches unterteilt. Diese werden zunächst in 1D-Arrays geglättet, anschließend linear embedded und äquivalent zu Token eines Textes verarbeitet. [7]

Im Gegensatz zu klassischen CNN, die explizit die zweidimensionalen räumlichen Zusammenhänge in Bildern ausnutzen, verzichten Vision Transformers auf diese Struktur. Stattdessen verlassen sie sich vollständig auf die Fähigkeit der Self-Attention, relevante

Muster und Zusammenhänge zu erkennen - unabhängig von der ursprünglichen räumlichen Anordnung. Da die Transformerarchitektur allerdings deutlich performanter als die CNN Architektur ist, kann durch das trainieren von sehr viel größeren Parameterzahlen (im Milliardenbereich) eine vergleichbare Leistung von CNN basierten Architekturen erreicht werden. Gleichzeitig profitieren Anwendungen von Vision Transformers stark von Transfer Learning. Sie sind besonders effektiv, wenn große vortrainierte Modelle auf spezifische Aufgaben wie die Handgestenerkennung feinjustiert werden. [7]

Nach Betrachtung der verschiedenen Modelle und Architekturen haben wir uns aus mehreren Gründen dafür entschieden *Mediapipe* zu nutzen. Zunächst fällt auf, dass, ausgenommen der Echtzeitanforderung, alle Modelle sehr ähnliche Leistungen erbringen können. Mediapipe hat den Vorteil, einen Input-Stream an Bildern effizient auswerten zu können und die Hand-Landmarks zu finden. Des Weiteren bietet Mediapipe sehr komfortable Pytgon-Pakete, die die Anwendung relativ einfach machen.

3.1.1 Mediapipe

Mediapipe ist eine Open-Source Plattform von Google, die mehrere Modelle zu verschiedenen Anwendungsbereichen, wie generativer KI, Bildverarbeitung, Tonverarbeitung und Textverarbeitung umfasst. Über die genaue Architektur und Technologie der einzelnen Modelle werden keine Details veröffentlicht.

Für die Handerkennung nutzt MediaPipe eine zweistufige Pipeline, bestehend aus einem Palmenerkennungsmodell und einem Hand-Landmark-Modell. Zunächst identifiziert das Palm Detection Model die Region, in der sich eine Hand befindet, und schneidet diesen Bereich aus dem Bild aus. Anschließend lokalisiert das Hand Landmark Model insgesamt 21 dreidimensionale Schlüsselpunktkoordinaten (Landmarks) innerhalb der erkannten Handregion. Diese Keypoints repräsentieren die Gelenke und Fingerknöchel der Hand und ermöglichen eine präzise Analyse von Handgesten.[1, 2]

Das Palm Detection Model basiert auf dem Prinzip der Single-Shot-Detection (SSD) (Abb. 3.2). Dabei handelt es sich um ein einzelnes, feedforward-basiertes Deep Neural

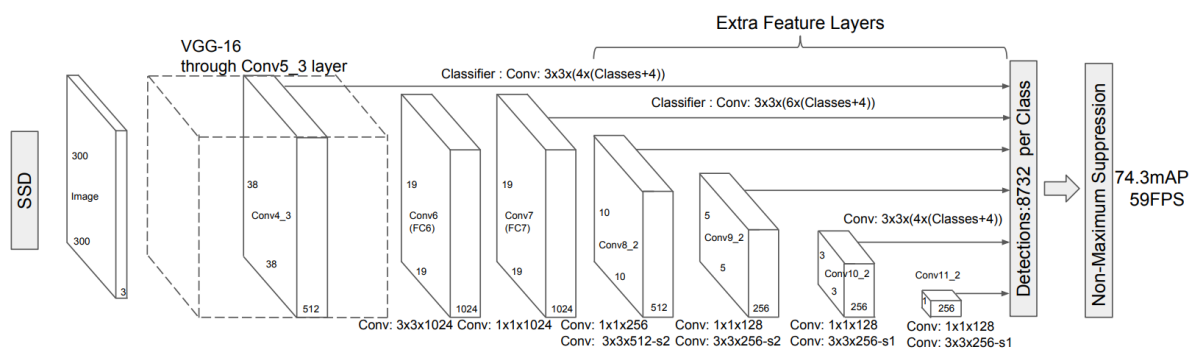


Abbildung 3.2: Mediapipe SSD Architektur aus [15]

Network, das ohne Resampling von Pixeln oder Features direkt Bounding Boxes vorher-sagt. Das Modell generiert eine feste Anzahl an Boxen und bewertet, ob sich die gesuchte Klasse, in diesem Fall eine Handfläche, innerhalb dieser Boxen befindet. Zur Auswahl der finalen Detektionen wird eine Non-Maximum Suppression angewendet.[15]

Das Hand Landmark Model führt anschließend eine direkte Regressionsvorhersage der 21

3D-Koordinaten durch. Die Trainingsdaten umfassen etwa 30.000 reale Bilder sowie synthetisch generierte Daten, um eine robuste Generalisierung zu gewährleisten. Trotz der Komplexität der Aufgabe ist das gesamte System so optimiert, dass es mit einer Latenz von nur etwa 17 Millisekunden auf mobilen Endgeräten ausgeführt werden kann, was den Einsatz in Echtzeitanwendungen ermöglicht.[1]

3.2 Zeitserienklassifikation

Die Daten die im zweiten Schritt, der Zeitreihenklassifikation, als Input dienen, sind mehrdimensional. Es gibt in jedem Zeitschritt 21 Punkte mit jeweils 3 Raumdimensionen. Da man diese Dimensionalität jedoch reduzieren könnte, wenn das dementsprechende Modell vielversprechend genug ist. Im folgenden werden einige populäre Methoden vorgestellt.

K-Means

Traditionelle Clustering-Algorithmen wie k-Means sind für die Analyse von Zeitserien nur bedingt geeignet, da sie zeitliche Abhängigkeiten und die Struktur der Daten über die Zeit hinweg nicht berücksichtigen. Um diesem Problem zu begegnen, wurde eine spezielle Variante namens TSkmeans entwickelt. Diese Methode erweitert den klassischen k-Means-Ansatz, indem sie Zeitstempel mit Gewichten versieht, wodurch die zeitliche Dimension explizit in die Analyse einbezogen wird. Die Iterationen verlaufen ähnlich wie beim herkömmlichen k-Means, jedoch werden die Gewichte über die gleichmäßig verteilte Zeitdimension geglättet. Zusätzlich berücksichtigt TSkmeans die Streuung innerhalb jeder einzelnen Zeitserie, was zu einer robusteren und aussagekräftigeren Clusterbildung führt. Diese Methode eignet sich nur für univariate Zeitserien, bei denen nur eine Variable über die Zeit hinweg beobachtet wird.[11]

Dynamic Time Warping

Ein weiterer Ansatz zur Analyse und zum Vergleich von Zeitserien ist die Verwendung des Nearest Neighbor Algorithmus in Kombination mit Dynamic Time Warping (DTW). DTW ermöglicht ein nicht-lineares Mapping zwischen zwei Zeitserien, wodurch Verschiebungen und Verzerrungen entlang der Zeitachse ausgeglichen werden können (Abb. 3.3).[13]

Dies geschieht mithilfe einer sogenannten Warping-Matrix, die die optimale Ausrichtung zweier Zeitreihen bestimmt. Dabei wird der Abstand zwischen jeweils zwei Punkten der Zeitserien berechnet und gewichtet - Punkte, die zeitlich näher beieinanderliegen, erhalten ein geringeres Gewicht. Auf diese Weise lassen sich auch Zeitserien vergleichen, die ähnliche Muster aufweisen, jedoch zeitlich versetzt oder unterschiedlich skaliert sind. Diese Methode eignet sich insbesondere für univariate Zeitserien.[13]

Shapelet Transform

Ein weiterer Ansatz zur Klassifikation von Zeitserien ist Shapelet Transform. Diese Methode ist eher für univariate Zeitserien geeignet. Shapelet Transform basiert auf der Idee,

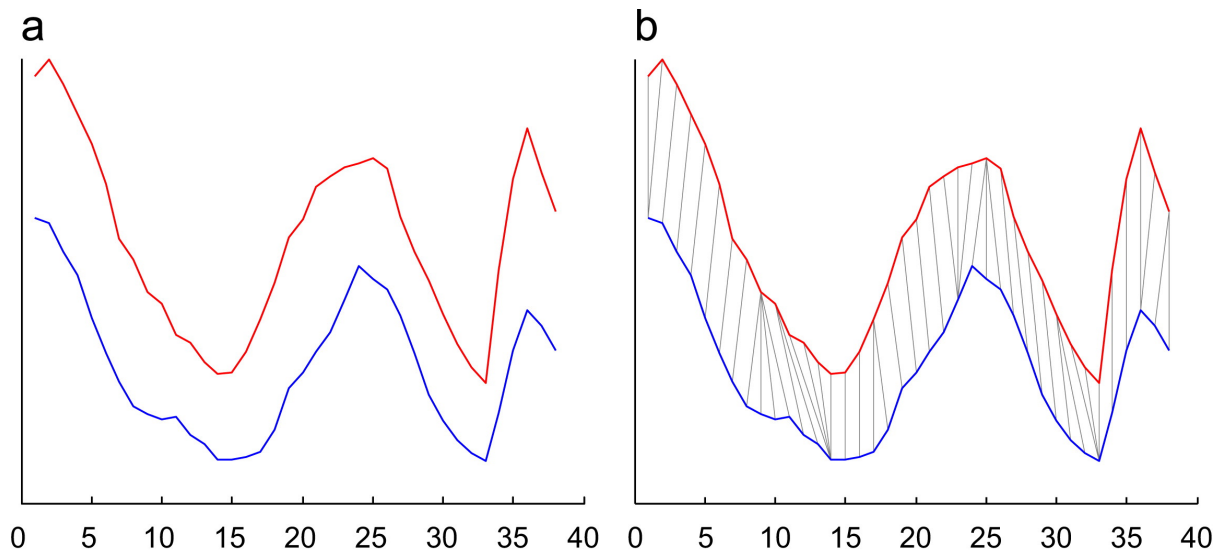


Abbildung 3.3: Veranschaulichung des Mappings durch DTW (aus [13])

dass sich charakteristische Muster, sogenannte Shapelets, innerhalb einer Zeitreihe identifizieren lassen. Shapelets sind kurze Ausschnitte aus der gesamten Zeitserie, die besonders aussagekräftig für die Unterscheidung zwischen Klassen sind. Diese Herangehensweise soll die menschliche Wahrnehmung von Mustern in Zeitreihen approximieren. Für die Klassifikation kommen Entscheidungsbäume zum Einsatz, die ein oder mehrere Shapelets finden, die eine möglichst gute Trennung der Klassen ermöglichen. Der Baum sucht dabei gezielt nach denjenigen Shapelets, die den größten Informationsgewinn liefern, um die gesamte Zeitserie korrekt zu klassifizieren.[24]

1D-CNN

1D-CNN stellen einen leistungsfähigen Ansatz zur Klassifikation von Zeitserien dar, sowohl im univariaten als auch im multivariaten Kontext. Ziel dieser Methode ist es, Störuschen in den Daten so weit wie möglich zu reduzieren, um klare und trennscharfe Klassen zu erhalten. Die 1D-Convolution funktioniert dabei analog zur bekannten 2D-Convolution aus der Bildverarbeitung, wobei die Filter entlang der Zeitachse gleiten und charakteristische Muster erkennen.[19]

Die zentrale Annahme ist, dass 1D-Convolutions in der Lage sind, relevante zeitliche Muster zu lernen und gleichzeitig Rauschen herauszurechnen. Allerdings ist zu beachten, dass Komponenten wie Bias, Batch Normalization und ReLU-Aktivierungsfunktionen das Rauschen nicht aktiv entfernen, sondern lediglich die Netzwerkausgabe beeinflussen. Ein besonders kritischer Faktor ist die Wahl der Kernel-Größe: Ist sie zu groß, kann dies zu einem überhöhten Rechenaufwand führen und zusätzlich Rauschen in die Zeitserie einbringen. Ist sie hingegen zu klein, werden wichtige Muster möglicherweise nicht erkannt.[19]

Für den Use-Case ist hier eine Erkenntnis wichtig: CNN sind in der Lage Zeitabhängigkeiten zu lernen.

MiniROCKET

MiniROCKET stellt eine nahezu deterministische Transformation für die Klassifikation von Zeitreihen dar. Die Methode zeichnet sich durch eine hohe Skalierbarkeit aus und basiert auf einem CNN mit fester Kernelgröße und festem Padding. Im Anschluss an die Transformation erfolgt die Klassifikation mittels eines linearen Klassifikators, was zu einer effizienten und gleichzeitig leistungsfähigen Modellarchitektur führt.[6]

LSTM-Netzwerke

Multivariate LSTM-Netzwerke eignen sich zur Verarbeitung multivariater Zeitreihen. Ein Ansatz kombiniert rekurrente neuronale Netze (RNNs) mit einem Attention-Mechanismus. Der Attention-Mechanismus besteht aus drei Attention-Layern sowie zwei Convolutional-Layer, die den Long Short-Term Memory (LSTM) Sequenz Encoder bilden. Dieser ermöglicht eine effektive Kodierung von Langzeit- und Kurzzeitabhängigkeiten innerhalb der Zeitreihen. Für die Modellierung ist eine Repräsentation der Daten in Form eines Graphen erforderlich, was einer Darstellung analog zu einem Bild mit mehreren Kanälen entspricht.[9]

Für den Use Case ist relevant, dass Zeitreihendaten analog zu einem Bild mit mehreren Kanälen aufgefasst werden können.

InceptionTime

InceptionTime stellt einen der frühesten und zugleich leistungsfähigsten Deep-Learning-Ansätze zur Klassifikation von Zeitreihen dar. Die Architektur basiert auf CNN und adaptiert das Inception-Modul, das ursprünglich für die Bildverarbeitung entwickelt wurde, auf den Zeitreihenkontext. Dabei kommen sogenannte Bottleneck-Schichten zum Einsatz, die die Anzahl der Parameter reduzieren und somit ein tieferes Modell ermöglichen. Zusätzlich werden Residual Connections integriert, um den Informationsfluss über tiefe Netzwerkschichten hinweg zu erleichtern und dem Problem des Vanishing Gradient entgegenzuwirken. Ein zentrales architektonisches Merkmal ist der Einsatz von Global Average Pooling, das zusätzlich zu Fully-Connected-Layers verwendet wird. Diese Technik reduziert die Anzahl der Parameter erheblich und verbessert gleichzeitig die Generalisierungsfähigkeit des Modells, was zu einer signifikanten Leistungssteigerung führt.[8]

Aufgrund der beobachteten Instabilität im Trainingsprozess wird InceptionTime nicht als einzelnes Modell, sondern als Ensemble aus fünf identisch strukturierten Netzwerken trainiert. Jedes dieser Netzwerke wird unabhängig voneinander optimiert, wodurch unterschiedliche Aspekte der zugrunde liegenden Daten erfasst werden können. Die finale Vorhersage erfolgt durch eine gewichtete Abstimmung der Einzelausgaben, was die Robustheit und Genauigkeit der Klassifikation deutlich erhöht und die Varianz bei der Accuracy verringert.[8]

Time Series Transformer

Der Time Series Transformer ist ein Modell, das speziell für die Analyse komplexer Zeitreihen entwickelt wurde. Es eignet sich besonders gut für Szenarien, in denen langfristige

Abhängigkeiten, nicht-deterministische Strukturen und komplexe Zusammenhänge über große zeitliche Distanzen eine Rolle spielen.

Das Modell kombiniert die klassische Transformer-Architektur, insbesondere das Attention-Mechanismus, mit Konzepten aus State-Space Models. Eine Besonderheit dieses Ansatzes ist, dass der Transformer probabilistisch gestaltet ist: Für jede Query-Matrix in einem Zeitschritt werden stochastische Samples aus einer parametrisierten Verteilung gezogen. Dadurch kann das Modell Unsicherheiten besser erfassen und flexibler auf die inhärente Stochastik vieler Zeitreihen reagieren.[18]

3.2.1 ST-GCN

Die Zeitreihe, die als Ergebnis des Mediapipe-Modells nun den Input zur Gestenklassifikation darstellt ist ein $V = [B \times D \times T \times P]$ Volumen. B ist der Batch und ausserhalb des Trainings gilt stets $B = 1$. D ist die Anzahl der Raumdimensionen, also $D = 3$. T ist die Länge der Zeitserie. Diese muss vor dem Training festgelegt werden. P ist die Anzahl der Landmarks. Demnach gilt bei der Handdetektion immer $P = 21$. Das Volumen ist also dreidimensional mit einer zusätzlichen Batch-Dimension, die nur für das Training benötigt wird $V = [1 \times 3 \times T \times 21]$.

Aus den betrachteten Modellen geht hervor, dass Zeitreihen äquivalent zu Bildern betrachtet werden können. Somit bieten sich CNN-Architekturen an. Zusätzlich haben Handgesten die Anforderung, dass die durch die Adjazenzmatrix festgelegten Zusammenhänge zwischen den Landmarks berücksichtigt werden müssen. Dieses Problem löst ein Spatial-Temporal Graph-Convolutional-Network (ST-GCN).

Die ST-GCN Architektur besteht hauptsächlich aus zwei Arten von Layern: Graph-Convolutions (GC) und Temporal Convolutions (TC). Die GC lernen die Zusammenhänge zwischen den Landmarks und die TC lernen die zeitlichen Zusammenhänge. Diese Layer werden abgewechselt. Am Ende des Netzwerks werden Fully Connected Layer genutzt um die Prediction zu treffen.

Die GC funktioniert ähnlich zu einem regulären linearen Layer. Es wird der Input $X \in \mathbb{R}^{P \times D}$ mit den Gewichten $W \in \mathbb{R}^{D \times C_{out}}$ multipliziert, wobei C_{out} die Anzahl der ausgehenden Kanäle des Layers sind. Allerdings wird die symmetrische Adjazenzmatrix $\hat{A} \in \mathbb{R}^{P \times P}$ berücksichtigt:

$$Z = \hat{A}XW$$

$Z \in \mathbb{R}^{P \times C_{out}}$ ist das veränderte Signal nach der GC.

Eine TC ist eine reguläre 2D-Convolution, nur dass die Größe des Kernels so gewählt wird, sodass dieser hauptsächlich über die Zeitdimension gleiten kann. Ein Beispiel ist ein $[9 \times 1]$ Kernel mit 4 Padding. Der Kernel ist also nicht quadratisch. Zwischen GC und TC kann ein Attention-Layer hinzugefügt werden, um die Ergebnisse weiter zu verbessern.[14, 17, 23]

4 Umsetzung

Die praktische Umsetzung des Use-Cases wird in Python implementiert. Die generelle Architektur ist in Abbildung 2.1 dargestellt.

Die Hand-Landmark-Recognition wird durch ein Mediapipe-Modell umgesetzt. Dieses Modell hat zwei verschiedene Konfigurationsmöglichkeiten. Der *IMAGE*-Modus ist die Standardeinstellung. In diesem Modus wird zuerst auf dem gesamten Bild die Bounding-Boxen der Hände gesucht, dieser Bereich ausgeschnitten und dort werden die Landmarks identifiziert. Da im Use-Case allerdings die Hände in aufeinanderfolgenden Frames eine ähnliche Position haben, gibe es noch einen *VIDEO*-Modus und einen *STREAM*-Modus. In diesen Modi gibt es ein kleineres Tracking-Modell, welches die Bounding-Box mit weniger Rechenaufwand findet, allerdings die Information benötigt, wo sich die Bounding-Box im vorherigen Frame befunden hat. Wenn dieses Modell kein Ergebnis findet, wird das reguläre Modell zur Suche der Bounding-Boxen verwendet. Dieser Ansatz spart einiges an Rechenleistung und ermöglicht so die Echtzeitanforderung. In der Implementierung wird der VIDEO-Modus verwendet.

Modus	FPS (experimentell ermittelt)
IMAGE	3
VIDEO	16
STREAM	14

Tabelle 4.1: Experimentell ermittelte minimale FPS innerhalb einer Minute Handdetektion. Zusätzlich zum Modell selbst benötigte die Pythonumgebung zur visuellen Darstellung und Kontrolle der Ergebnisse minimalen Rechenaufwand.

Das Ergebnis der Landmark-Detektion ist ein Array mit 21 3D-Punkten. Demnach ist jeder Frame ein Volumen $V_0 = [3 \times 21]$. Eine Geste ist eine Zeitserie aus V_0 mit der Länge $T = 30$. somit ergibt sich ein Volumen $V = [B \times 3 \times 30 \times 21]$. Dieses Volumen dient als Input für das Klassifikationsmodell. Hierfür wurde ein ST-GCN trainiert.

Das Ergebnis der Klassifikation löst einen Befehl für Powerpoint aus, wenn eine bestimmte Mindestwahrscheinlichkeit vorliegt.

4.1 Projektaufbau

Die Kamera wird mithilfe von *CV2* ausgelesen. Das Bild wird in ein Format umgewandelt, dass das Mediapipe-Modell verwendet.

```
import cv2
import numpy as np

cam = cv2.VideoCapture(0)
while True:
```

```
_,frame = cam.read()
frame = cv2.cvtColor(frame,cv2.COLOR_BGR2RGB).copy().astype(np.uint8)
```

Der Frame muss kopiert werden, da `cam.read()` Frames als schreibgeschützt markiert. Des weiteren muss die Repräsentation von BGR zu RGB und in 8-Bit Integer umgewandelt werden.

Das Frame wird dann dem Mediapipe-Modell übergeben.

```
import mediapipe as mp
from mediapipe.tasks.python import vision

vision_model = mp.tasks.vision.GestureRecognizer.create_from_options(
    vision.GestureRecognizerOptions(
        base_options = mp.tasks.BaseOptions(model_asset_path=PATH),
        running_mode=mp.tasks.vision.RunningMode.VIDEO,
        num_hands=1,
    )
)

def recognize_hand(frame,timestamp):
    mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=frame)
    return vision_model.recognize_for_video(mp_img,timestamp)
```

Es ist ein rotierender Puffer implementiert, der immer genau die 30 Ergebnisse für das Volumen der Klassifikation speichert. Konkret ist es einfach ein Array, mit dem ein Stapel implementiert ist. Dieser Puffer wird, insofern er für jeden Zeitschritt gefüllt ist, in einen Tensor umgewandelt und dem Klassifikationsmodell mitgegeben. Das klassifikationsmodell ist mit *pytorch* implementiert.

```
import torch
from torch import Tensor

def convert_to_tensor(data:dict)->Tensor:
    output = []
    for timestep in data.values():
        landmarks = timestep.hand_landmarks[0]
        skeleton = []
        for point in landmarks:
            skeleton.append([point.x,point.y,point.z])
        output.append(skeleton)

    # permute to get correct shape (T,P,D)-->(D,T,P);
    return torch.as_tensor(output).permute(2,0,1)
```

Wenn die Wahrscheinlichkeit der vorhersagten Klasse ein Limit übersteigt, wird ein Befehl an Powerpoint gesendet. Dazu werden die Logits, die das Modell zurückgibt, mittels der Softmax-Funktion in Wahrscheinlichkeiten umgewandelt, die aufsummiert 100% ergeben.

```

import torch
import torch.nn.functional as F
import ST_GCN

classification_model = ST_GCN(3)
classification_model.load_state_dict(torch.load(PATH, weights_only=True))

def classify_series(tensor, min_confidence=0.65) -> str:
    probabilities = F.softmax(classification_model(tensor), dim=1)
    max_probability, idx = torch.max(probabilities, dim=1)
    if max_probability.item() > min_confidence:
        # This is a dict to match the index to a string
        return REVERSE_LOOKUP[idx.item()]
    else:
        return "No class"

```

Die Befehle werden mithilfe einer Powerpointintegration von *comtypes* und *pyautogui* umgesetzt. Dazu wurde eine eigene Klasse implementiert, um die Details zur Steuerung zu abstrahieren und die nötigen Fehler zu behandeln.

```

presentation = PowerPoint(PATH)
while True:
    # hier werden die oben beschreiben Schritte durchgeführt
    # [...]
    classification = classify_series(tensor)
    match classification:
        case "gesture_backward":
            presentation.return_slide()
        case "gesture_forward":
            presentation.advance_slide()
        case "gesture_blacken":
            presentation.toggle_blacken()
        case _:
            pass

```

Damit nicht unbeabsichtigt mehrere Befehle hintereinander ausgeführt werden, wird das Ausführen von Befehlen für 5 Sekunden deaktiviert, nachdem ein Befehl erfolgreich ausgeführt wurde.

4.2 Training

Das Mediapipe-Modell zur Landmark-Detektion kann out-of-the-box und ohne Training zur Feinjustierung verwendet werden.

Das ST-GCN zur Klassifikation wird von Grund auf aufgebaut und trainiert. Dazu wurden zunächst drei Gesten festgelegt. Jeweils eine Geste für das Springen auf die nächste, beziehungsweise vorherige Folie. Die dritte Geste ist zum Schwarz- und Freischalten

der Präsentation. Für das Training wurden von jeder Geste 100 Trainingsvideos aufgenommen. Jedes der vier Teammitglieder hat 25 Videos pro Geste beigesteuert, um eine bessere Generalisierbarkeit des Modells zu fördern. Jedes Video wurde mithilfe des Mediapipe-Modells in einen Tensor mit dem Volumen V umgewandelt. Diese Tensoren wurden mithilfe des *pickle* Moduls serialisiert. Das spart im Vergleich zu den Videos Speicherplatz und beschleunigt den Trainingsprozess des Klassifikationsmodells, da das Mediapipe Modell dann nicht mehr laufen muss.

Da der erzeugte Datensatz mit 300 Einträgen zu klein sein könnte, um ein gut generalisierbares Modell zu trainieren, wird während des Trainings Rauschen eingeführt. Das entspricht einer Vergrößerung des Datensatzes. Da das Volumen V die gleiche Anzahl an Features hat, wie ein RGB-Bild, kann hierzu die *torchvision*-Bibliothek genutzt werden.

```
import torchvision.transforms as transforms
import torchvision.transforms.v2 as transformsv2

# in der Dataset Klasse
self.transform = transforms.Compose(
    [
        transformsv2.GaussianNoise(
            mean=0, sigma=0.025, clip=True
        ) # clip to keep values in range [0..1]
    ]
)
```

Abbildung 4.1 zeigt, dass das Einführen von Rauschen den Trainingserfolg verbessern kann.

Zusätzlich zum Rauschen sollen Dropout-Layer ein Overfitting vermeiden.

4.3 Modellarchitektur

Die genaue Architektur des Mediapipe-Modells wird nicht veröffentlicht.

Das Klassifizierungsmodell wurde von uns implementiert und besteht aus zwei GC-Layers, zwei TC-Layers und zwei linearen Layers. Das Modell kennt die Adjazenzmatrix zu den Landmarks der Hand. Zwischen den Layers wird die ReLU-Aktivierungsfunktion genutzt. Die gesamte Architektur ist in Abbildung 4.2 dargestellt.

Graph Convolution

Die GC wird nach der im Grundlagenteil beschriebenen Vorschrift implementiert. Allerdings ist diese GC nur für zweidimensionale Daten anwendbar. Deshalb muss bei dem Inputvolumen V pro Batch durch die Zeitdimension iteriert werden und die GC für jeden Zeitschritt einzeln durchgeführt werden. Danach wird das Volumen wieder hergestellt. Der folgende Code ist nur die *forward()* Funktion des GC-Layers und geht davon aus, dass immer $B = 1$ gilt.

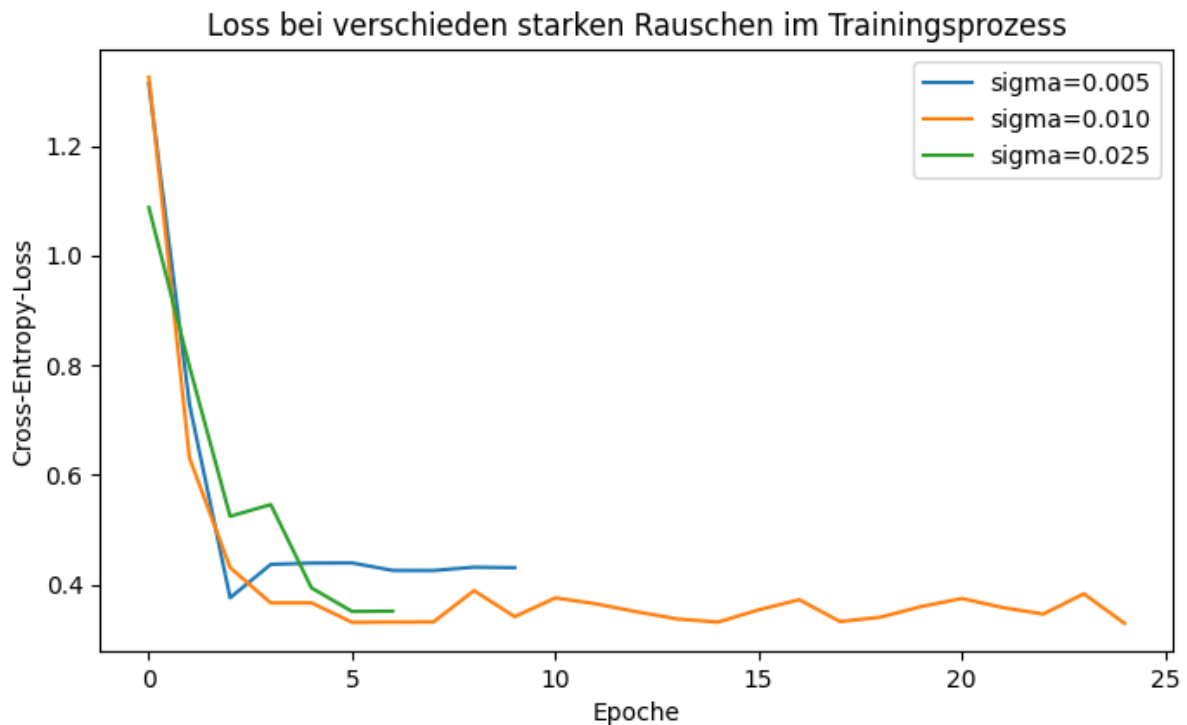


Abbildung 4.1: Darstellung des Cross-Entropy-Loss im Training des Klassifizierungsmodells bei verschiedenen starken Rauschen auf den Trainingsdaten

```
import torch

def forward(self, input, adj):
    # expected input shape (B,D,T,P)
    B,D,T,P = input.shape
    output = []

    # iterieren durch die Zeitdimension
    for t in range(T):
        x_t = input[:, :, t, :]
        x_t = torch.matmul(x_t, adj)
        x_t = torch.matmul(self.weight, x_t)
        if self.bias is not None:
            x_t = x_t + self.bias
        output.append(x_t)

    # Zusammenfügen des Tensors
    # und die Reihenfolge der Dimensionen sortieren
    return torch.stack(output).permute(1,2,0,3)
```

Temporal Convolution

Die TC sind in der Implementierung normale 2D-Convolutions. Der Kernel ist nicht quadratisch, sondern in die Zeitdimension gestreckt und hat die Form (9,1). Damit das Volumen V nicht verzerrt wird, muss ein unsymmetrisches Padding von (4,0) hinzugefügt

werden. Da der Kernel in einer Dimension nur 1 ist, wird der Stride auf 1 gesetzt. Eine beispielhafte Konfiguration eines 2D-Convolutional Layers sieht folgendermaßen aus:

```
import torch.nn as nn

temporal_convolution = nn.Conv2d(
    in_channels=32, out_channels=64, kernel_size=(9,1),
    stride=1, padding=(4,0)
)
```

Linear Layer

Am Ende des ST-GCN sind 2 Linear Layer. Diese dienen dem Zweck, den mehrdimensionalen Tensor auf die gewünschten Klassen zu mappen. Da das Ergebnis Logits sind, muss ein Softmax angewendet werden, um die Wahrscheinlichkeiten der einzelnen Vorhersagen zu erhalten. Beim Softmax addieren sich die Wahrscheinlichkeiten aller Vorhersagen auf 100%.

Verbunden ergeben diese Layer die Gesamtarchitektur des ST-GCN (Abb. 4.2).

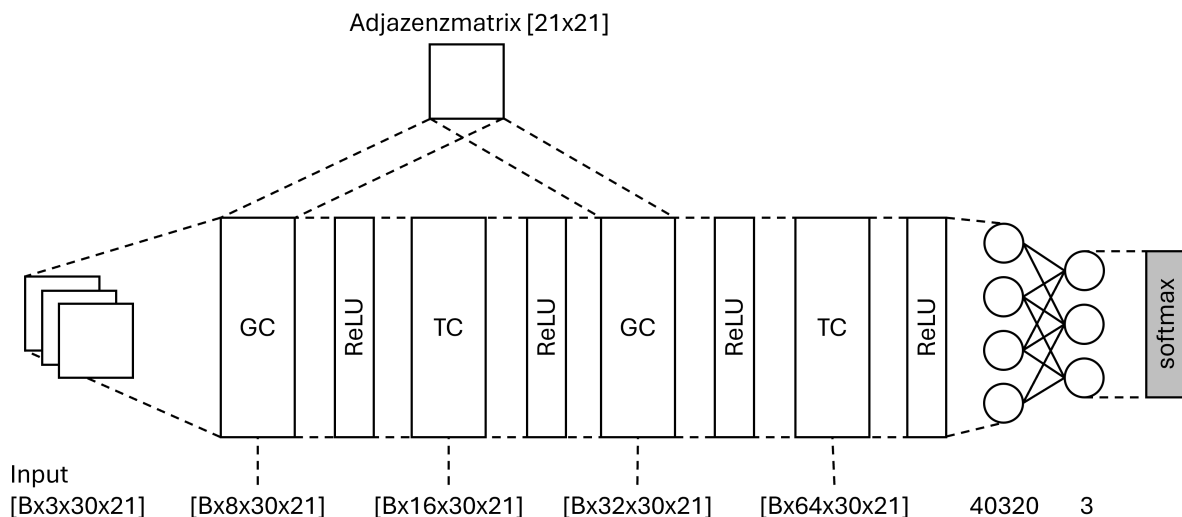


Abbildung 4.2: Die Architektur, die das ST-GCN verwendet. Die TC sind reguläre 2D-Convolutionals mit einem Kernel von (9,1), vertikalem Padding von jeweils 4 und einem Stride von 1. Die Beschriftung unter dem Modell beschreibt jeweils die Form des Tensors, den der Layer ausgibt.

4.4 Ergebnis

Beim Training haben wir die Cross-Entropy-Loss-Funktion genutzt. Da wir 3 Klassen haben, würde unser Modell bei einem Loss von $\ln(2) = 1,1$ die Klassen zufällig raten[12]. Das Modell, das wir ausgewählt haben hat einen deutlich niedrigeren Loss von 0.32 (Abb. 4.1 - sigma=0,010). Auch die anderen Performance-Metriken in Tabelle 4.2 zeigen, dass das Modell ein gutes Ergebnis liefert.

Metrik	Ergebnis
Cross-Entropy-Loss	0.32
Accuracy	85,48%
Precision	86,73%
Recall	85,48%

Tabelle 4.2: Performance-Metriken des Modells. Sie wurden auf einem Subset mit 20% des gesamten Datensatzes mit der *scikit-learn*-Bibliothek ermittelt.

Ein Problem, das beim Nutzen der Anwendung auffällt, ist dass die Gesten vor dem Training zu lange gewählt wurden. Das Programm geht von Gesten mit 30 Frames aus. Bei einer Framerate, die zwischen 5 und 10 FPS schwankt, dauert es zwischen 3 und 6 Sekunden bis eine Geste erkannt und in einen Befehl umgesetzt wird. In Hinblick auf die Benutzererfahrung des Programms, müsste die Gestenlänge auf maximal 1 Sekunde reduziert werden. Damit erhöht sich allerdings das Risiko, dass nebensächliche Bewegungen als Geste gewertet werden. Ebenso fällt bei genauerem Testen auf, dass die SSchwarzschaltenünd die "Folie zurück"Geste noch relativ häufig verwechselt werden.

5 Ausblick

Ein zentrales Problem des aktuellen Programms besteht darin, dass die Gestenerkennung zu lange dauert, was sich negativ auf das Nutzungserlebnis auswirkt. Die Ursache liegt in der Kombination mehrerer Faktoren: Die Framerate liegt unter 10 Bildern pro Sekunde, wobei jede Geste aus 30 aufeinanderfolgenden Frames besteht. Hinzu kommen eine Reaktionszeit von etwa 0,5 Sekunden sowie eine Cooldown-Phase von 5 Sekunden. Erschwerend kommt hinzu, dass die Sequenz von 30 Frames bei einer fehlerhaften Detektion unterbrochen werden kann, was den Prozess zusätzlich verlängert und unzuverlässig macht.

Ziel einer Weiterentwicklung ist es daher, die Benutzererfahrung durch eine effizientere Gestenerkennung deutlich zu verbessern. Zwei zentrale Ansätze können dabei verfolgt werden: Erstens sollte durch eine Erhöhung der Framerate und eine Verkürzung der Gestenlänge die Reaktionsgeschwindigkeit optimiert werden. Hierzu wird eine neue Architektur vorgeschlagen, bei der zunächst ein Bounding Box-Detektor die Hand lokalisiert. Anschließend entscheidet ein leichtgewichtiges Modell, ob ein Frame für die Gestenerkennung relevant ist. Irrelevante Frames werden sofort verworfen, während relevante an ein Landmark-Detektionsmodell und dem Klassifizierungsmodell zugeführt werden.

Zweitens sollte angestrebt werden, die Gesten selbst zu verkürzen, indem nicht mehr die gesamte Zeitreihe klassifiziert wird, sondern nur noch sogenannte Key-Poses. Dies erfordert eine Umgestaltung des gerade genannten Modells zur Relevanzbewertung zu einem Key-Frame-Detektor, der gezielt Schlüsselmomente innerhalb der Gesten identifiziert und zur Klassifikation heranzieht. Auf diese Weise kann die Gestenerkennung nicht nur beschleunigt, sondern auch robuster und benutzerfreundlicher gestaltet werden.

6 Fazit

Der Use-Case einen digitalen Presenter zu implementieren kann mithilfe von einem hybriden Modell implementiert werden. Unsere Architektur besteht aus einem vortrainierten Mediapipe-Landmark-Detektors und einem eigens implementierten ST-GCN. Hierfür wurde ein von Grund auf programmierter GC-Layer mit klassischen 2D-Convolutional Layers verbunden. Im Anschluss wurde ein eigenes Trainingsset aufgenommen und aufgearbeitet. Um ein stabileres Training zu erhalten wird mit Rauschen und Dropout-Layern gearbeitet. Der Vergleich zeigt, dass moderates Rauschen den Trainingserfolg verbessert. Der Prototyp kann Gesten relativ zuverlässig erkennen und eine Powerpoint-Präsentation ansteuern. Allerdings hat das Programm einige Probleme bei der Benutzerfreundlichkeit. Vor Allen die Dauer, bis eine Geste erkannt wird, ist zu lange, um eine ganze Präsentation zu halten. Im Ausblick schlagen wir einige Ansätze vor, um diese Probleme zu beheben.

Literaturverzeichnis

- [1] Google (Mediapipe). *Hand landmark model bundle*. URL: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer#hand_landmark_model_bundle (besucht am 07.05.2025).
- [2] Google (Mediapipe). *Vision Examples*. URL: <https://ai.google.dev/edge/mediapipe/solutions/examples> (besucht am 07.05.2025).
- [3] Puja Bharati und Ankita Pramanik. „Deep Learning Techniques—R-CNN to Mask R-CNN: A Survey“. In: *Computational Intelligence in Pattern Recognition*. Hrsg. von Asit Kumar Das u. a. Singapore: Springer Singapore, 2020, S. 657–668. ISBN: 978-981-13-9042-5.
- [4] Z. Cao u. a. „OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2019).
- [5] Zhe Cao u. a. „Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields“. In: *CVPR*. 2017.
- [6] Angus Dempster, Daniel F. Schmidt und Geoffrey I. Webb. „MINIROCKET: A Very Fast (Almost) Deterministic Transform for Time Series Classification“. In: *CoRR* abs/2012.08791 (2020). arXiv: 2012.08791. URL: <https://arxiv.org/abs/2012.08791>.
- [7] Alexey Dosovitskiy u. a. „An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale“. In: *CoRR* abs/2010.11929 (2020). arXiv: 2010.11929. URL: <https://arxiv.org/abs/2010.11929>.
- [8] Hassan Ismail Fawaz u. a. „InceptionTime: Finding AlexNet for Time Series Classification“. In: *CoRR* abs/1909.04939 (2019). arXiv: 1909.04939. URL: <http://arxiv.org/abs/1909.04939>.
- [9] En Fu u. a. „Temporal self-attention-based Conv-LSTM network for multivariate time series prediction“. In: *Neurocomputing* 501 (2022), S. 162–173. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2022.06.014>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231222007330>.
- [10] Andrew Howard u. a. *Searching for MobileNetV3*. 2019. arXiv: 1905.02244 [cs.CV]. URL: <https://arxiv.org/abs/1905.02244>.
- [11] Xiaohui Huang u. a. „Time series k-means: A new k-means type smooth subspace clustering for time series data“. In: *Information Sciences* 367-368 (2016), S. 1–13. ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2016.05.040>. URL: <https://www.sciencedirect.com/science/article/pii/S0020025516303796>.
- [12] Chris Hughes. *A Brief Overview of Cross Entropy Loss*. Sep. 2024. URL: <https://medium.com/@chris.p.hughes10/a-brief-overview-of-cross-entropy-loss-523aa56b75d5> (besucht am 09.07.2025).

- [13] Young-Seon Jeong, Myong K. Jeong und Olufemi A. Omitaomu. „Weighted dynamic time warping for time series classification“. In: *Pattern Recognition* 44.9 (2011). Computer Analysis of Images and Patterns, S. 2231–2240. ISSN: 0031-3203. DOI: <https://doi.org/10.1016/j.patcog.2010.09.022>. URL: <https://www.sciencedirect.com/science/article/pii/S003132031000484X>.
- [14] Thomas N. Kipf und Max Welling. „Semi-Supervised Classification with Graph Convolutional Networks“. In: *CoRR* abs/1609.02907 (2016). arXiv: 1609.02907. URL: <http://arxiv.org/abs/1609.02907>.
- [15] Wei Liu u.a. „SSD: Single Shot MultiBox Detector“. In: *CoRR* abs/1512.02325 (2015). arXiv: 1512.02325. URL: <http://arxiv.org/abs/1512.02325>.
- [16] Tomas Simon u.a. „Hand Keypoint Detection in Single Images using Multiview Bootstrapping“. In: *CVPR*. 2017.
- [17] Jae-Hun Song, Kyeongbo Kong und Suk-Ju Kang. „Dynamic Hand Gesture Recognition Using Improved Spatio-Temporal Graph Convolutional Network“. In: *IEEE Transactions on Circuits and Systems for Video Technology* 32.9 (2022), S. 6227–6239. DOI: 10.1109/TCSVT.2022.3165069.
- [18] Binh Tang und David S Matteson. „Probabilistic Transformer For Time Series Analysis“. In: *Advances in Neural Information Processing Systems*. Hrsg. von M. Ranzato u.a. Bd. 34. Curran Associates, Inc., 2021, S. 23592–23608. URL: https://proceedings.neurips.cc/paper_files/paper/2021/file/c68bd9055776bf38d8fc43c0ed283Paper.pdf.
- [19] Wensi Tang u.a. „Rethinking 1d-cnn for time series classification: A stronger baseline“. In: *arXiv preprint arXiv:2002.10061* (Feb. 2020), S. 1–7.
- [20] OpenCV - Open Source Computer Vision. *Cascade Classifier*. URL: https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html (besucht am 06.07.2025).
- [21] Weiyue Wang und Ulrich Neumann. „Depth-aware CNN for RGB-D Segmentation“. In: *ECCV*. Computer Vision Foundation, 2018. URL: <https://link.springer.com/conference/eccv>.
- [22] Shih-En Wei u.a. „Convolutional pose machines“. In: *CVPR*. 2016.
- [23] Sijie Yan, Yuanjun Xiong und Dahua Lin. „Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition“. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (Apr. 2018). DOI: 10.1609/aaai.v32i1.12328. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/12328>.
- [24] Willian Zalewski u.a. „Exploring shapelet transformation for time series classification in decision trees“. In: *Knowledge-Based Systems* 112 (2016), S. 80–91. ISSN: 0950-7051. DOI: <https://doi.org/10.1016/j.knosys.2016.08.028>. URL: <https://www.sciencedirect.com/science/article/pii/S0950705116303033>.
- [25] Fan Zhang u.a. *MediaPipe Hands: On-device Real-time Hand Tracking*. 2020. arXiv: 2006.10214 [cs.CV]. URL: <https://arxiv.org/abs/2006.10214>.